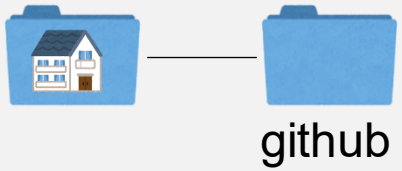
演習：Gitの応用操作

慶應義塾大学理工学部物理情報工学科
渡辺

今回の目標

- git amendによりコミットが変更されることを確認する
- git mergeの衝突を解決する
- git rebaseにより歴史を改変する
- git rebaseの衝突を解決する
- git bisectを使ってみる

課題1 – Step 1



ホームディレクトリ

Git管理された
ディレクトリ

リポジトリのクローン

```
cd
cd github
git clone https://github.com/cpss2026-git/amend-sample.git
cd amend-sample
```

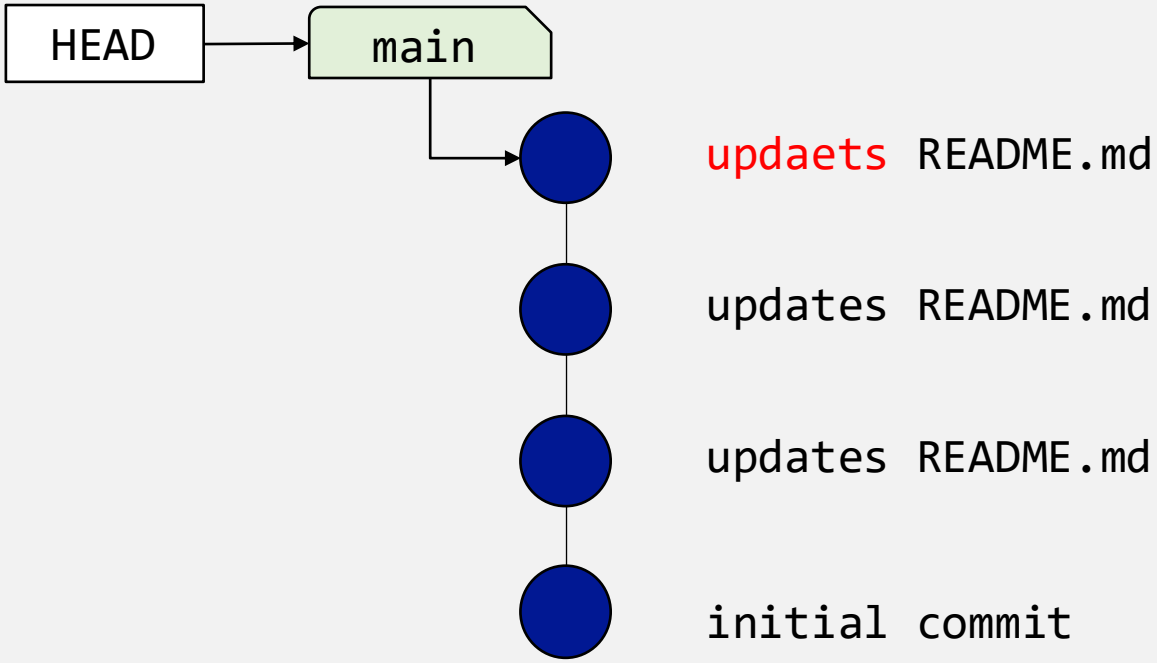
ここがカレントディレクトリに

github　amend-sample

課題1 – Step 2

歴史の確認

```
git log --oneline
```

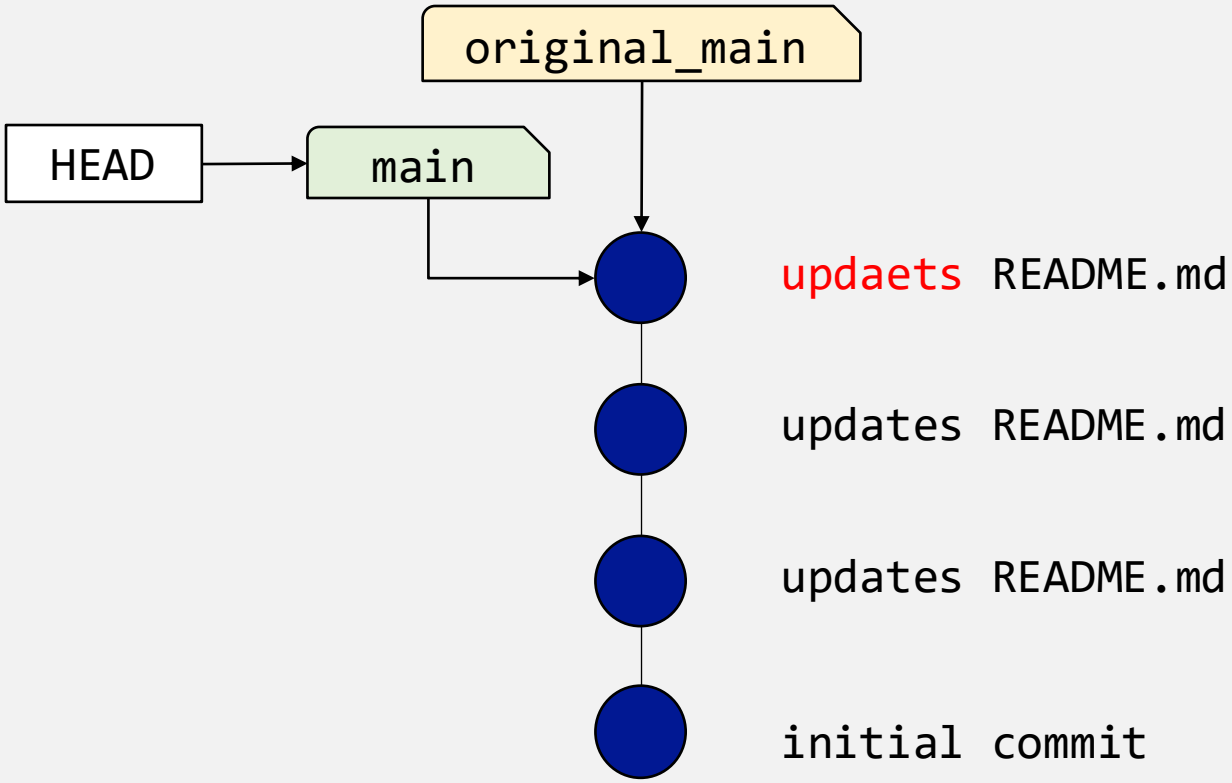


課題1 – Step 3

コミットの保存

現在のコミットに別名をつけておく

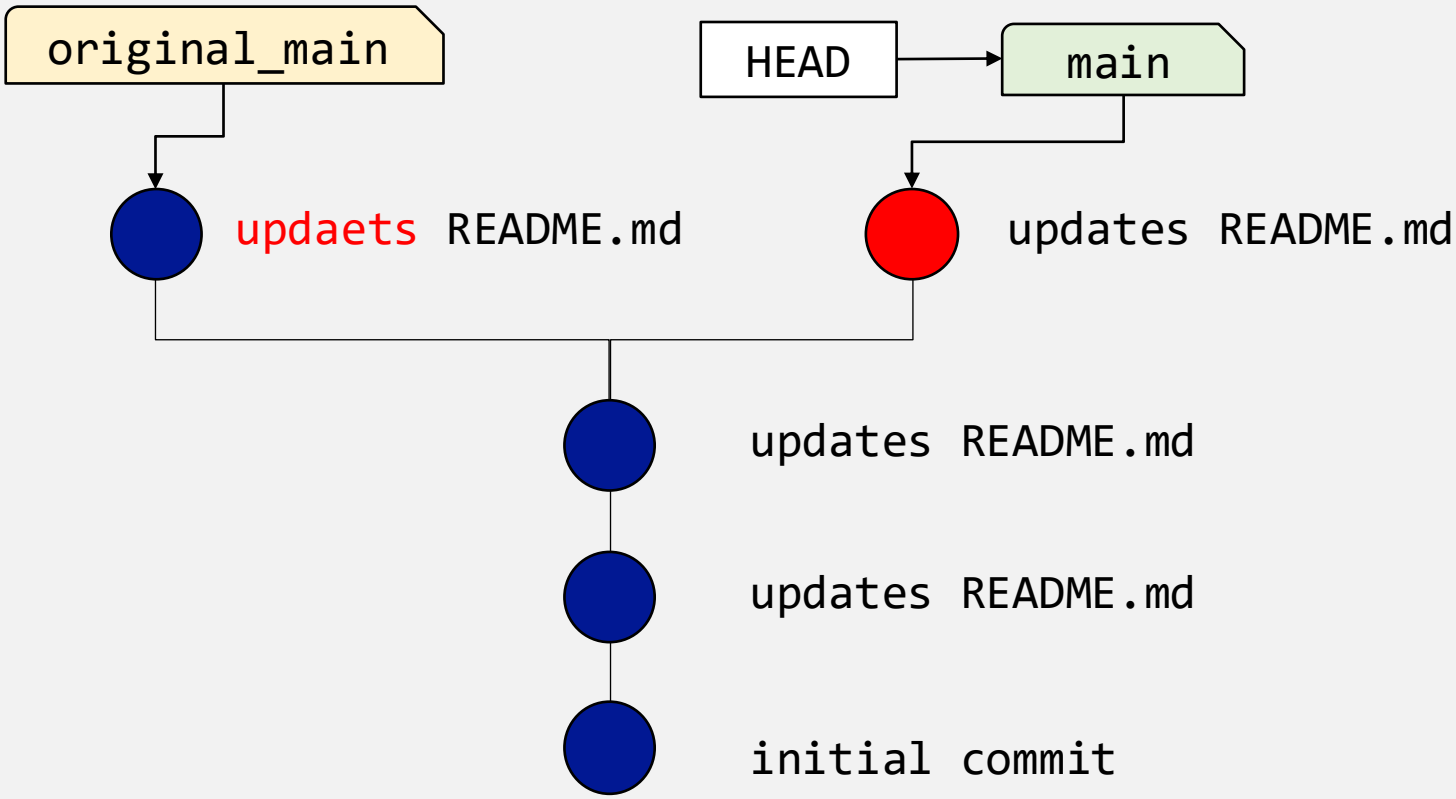
```
git branch original_main
```



課題1 – Step 4

コミットメッセージの修正

```
git commit --amend -m "updates README.md"
```

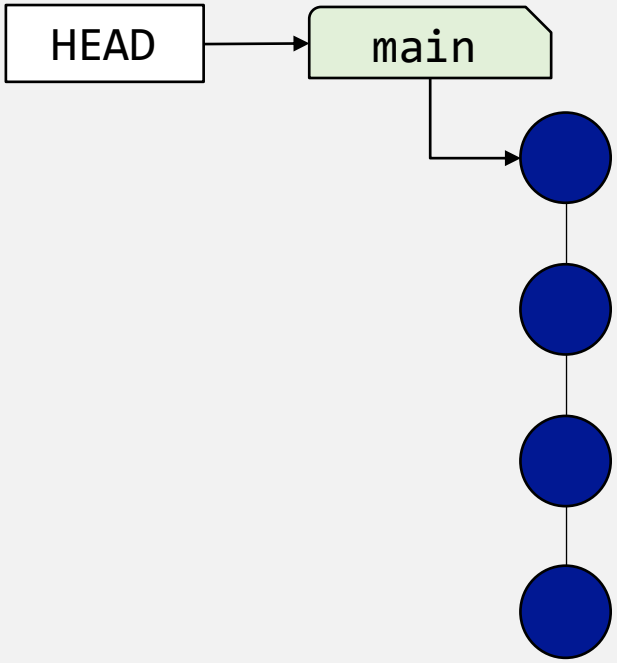


課題1 – Step 5

歴史の確認

```
git log --oneline
```

修正されていることを確認

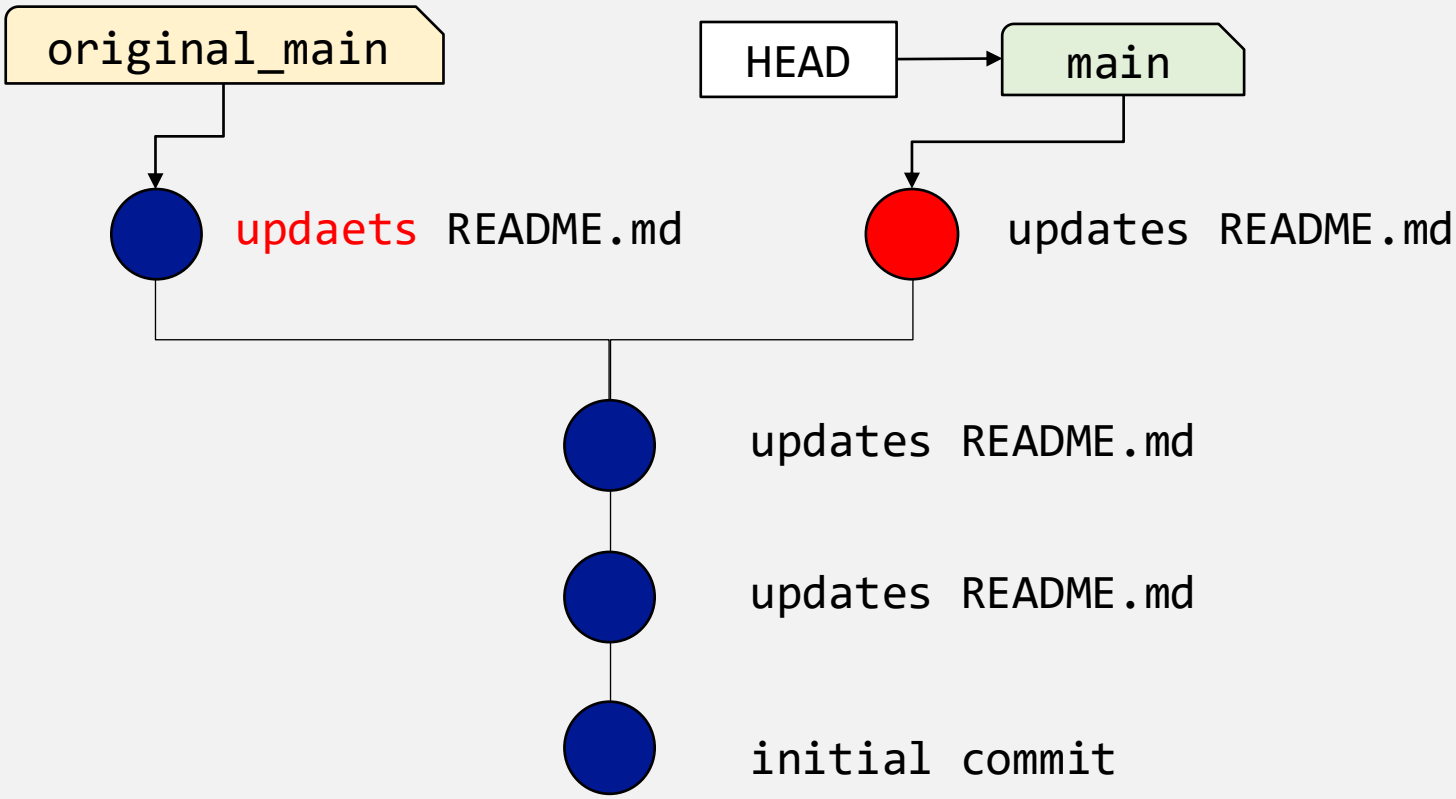



```
updates README.md

updates README.md

updates README.md

initial commit
```

課題1 – Step 6

歴史が分岐したことを確認

```
git log --all --graph --oneline
```



課題2 – Step 1

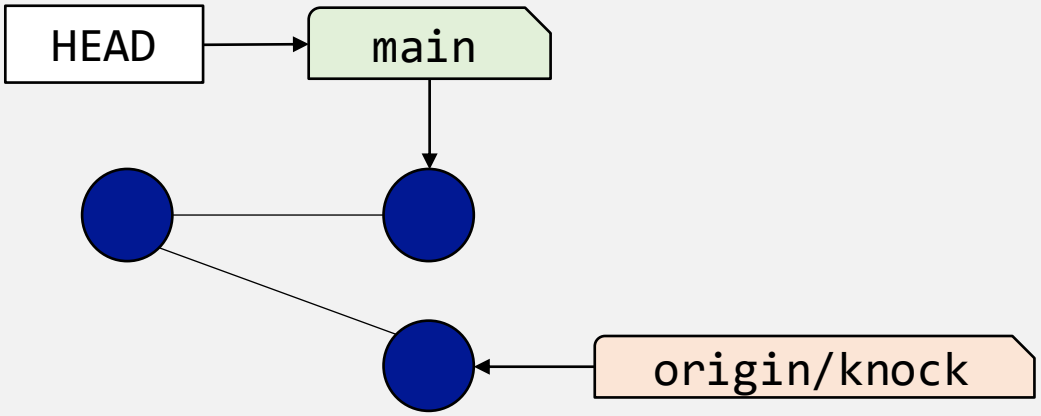
リポジトリのクローン

```
cd
cd github
git clone https://github.com/cpss2026-git/merge-sample.git
cd merge-sample
```

課題2 – Step 2

ブランチの確認

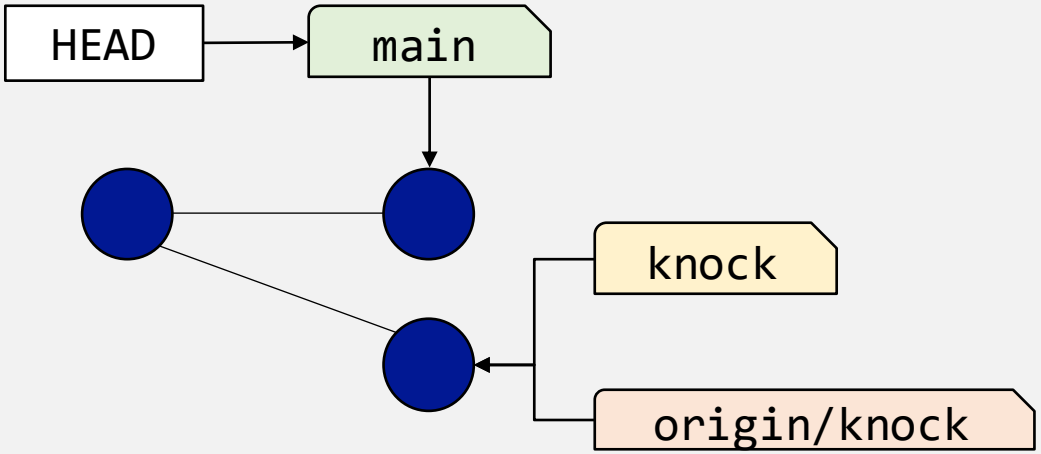
```
git branch -vva
```



ブランチの作成

```
git branch knock origin/knock
```

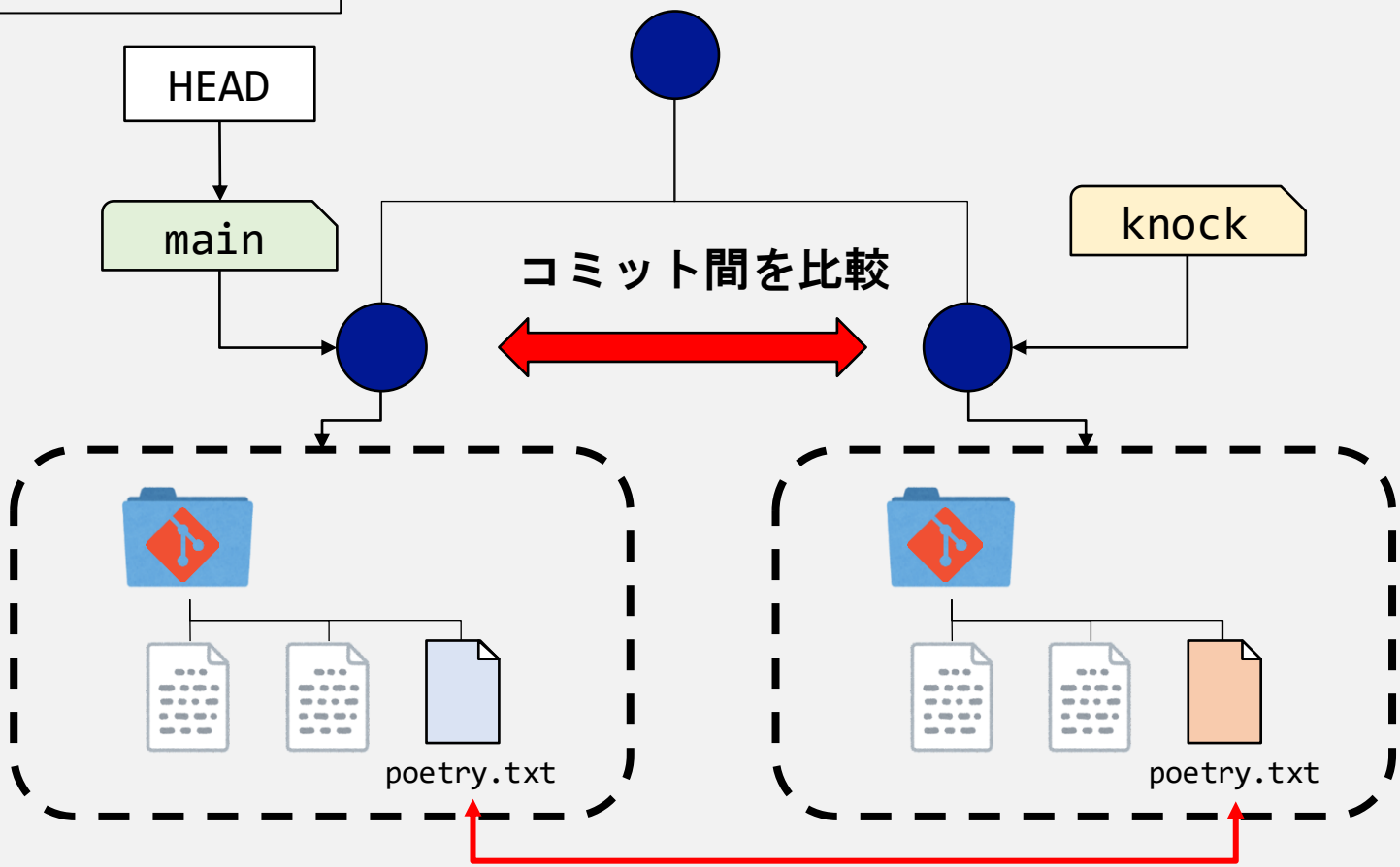
origin/knockの指すコミットに
knockブランチを作成



課題2 – Step 3

差分確認

```
git diff knock
```

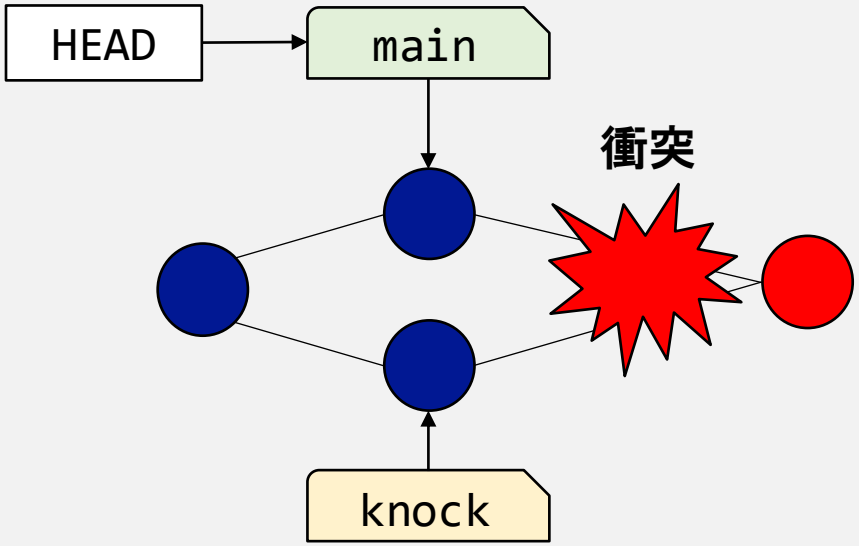


ここに差分を検出

課題2 – Step 4

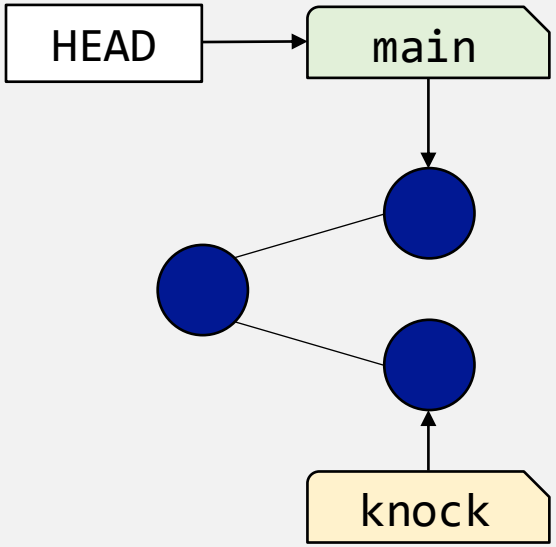
マージ

```
git merge knock
```



マージの中断

```
git merge --abort
```



このプロセスでpoetry.txtが
どのように変化したか調べる

課題2 – Step 5

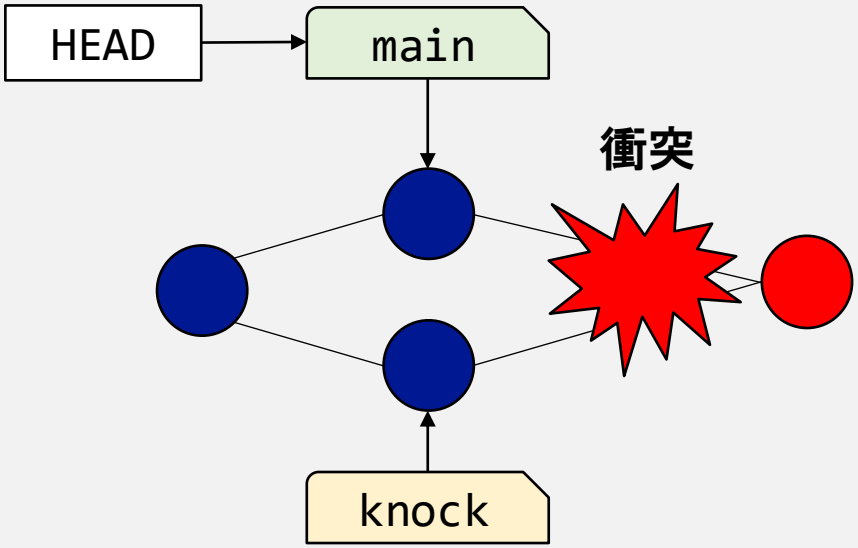
マージ

```
git merge knock
```

potery.txtをエディタで修正

衝突の解決とコミット

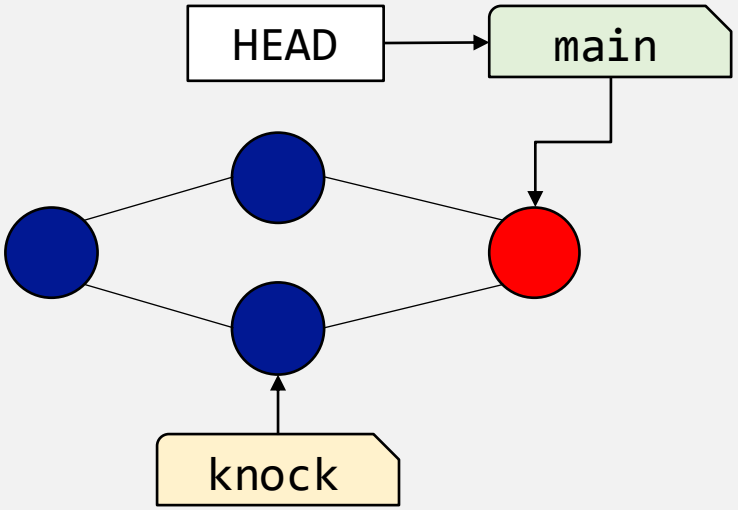
```
git add poetry.txt
git commit -m "knock"
```



課題2 – Step 6

マージされた「歴史」を確認

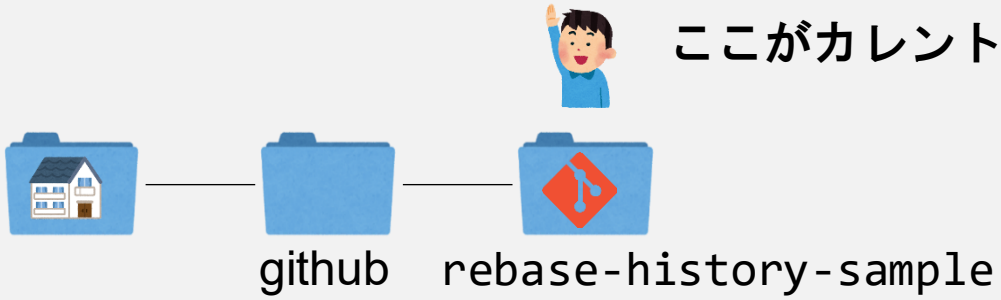
```
git log --all --graph --oneline
```



課題3 – Step 1

リポジトリのクローン

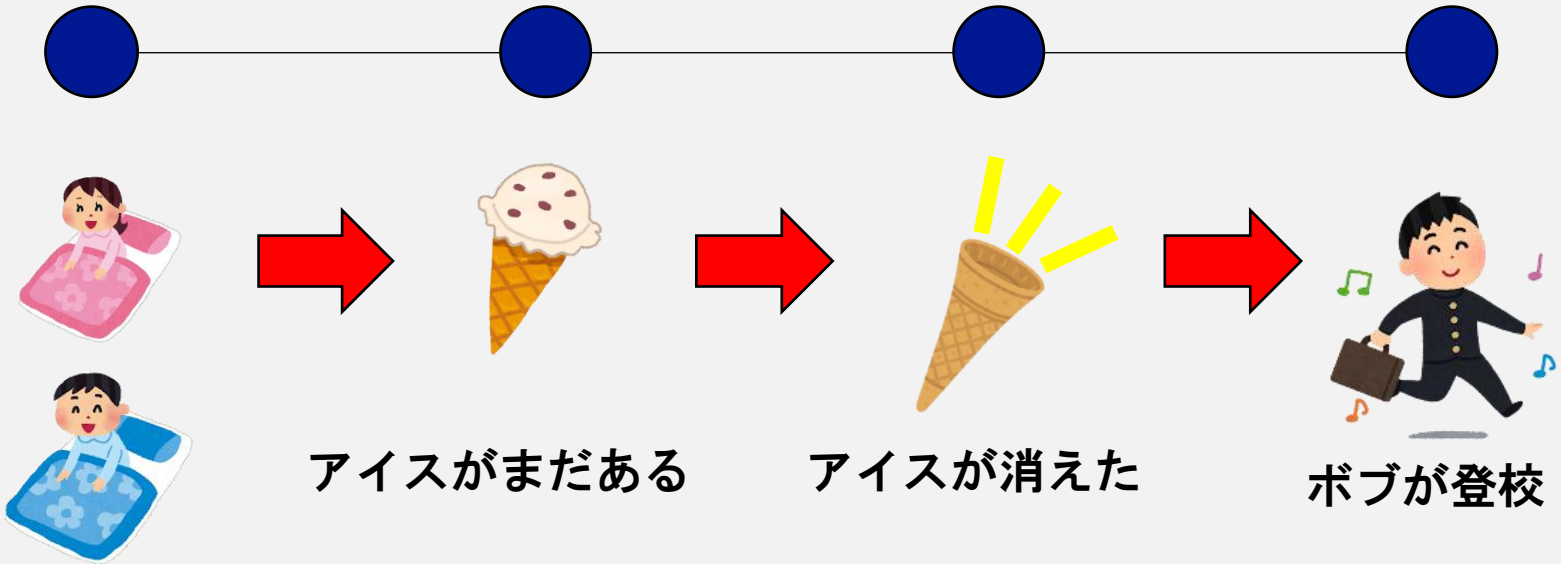
```
cd
cd github
git clone https://github.com/cpss2026-git/rebase-history-sample.git
cd rebase-history-sample
```



課題3 – Step 2

歴史の確認

```
git log --oneline
```

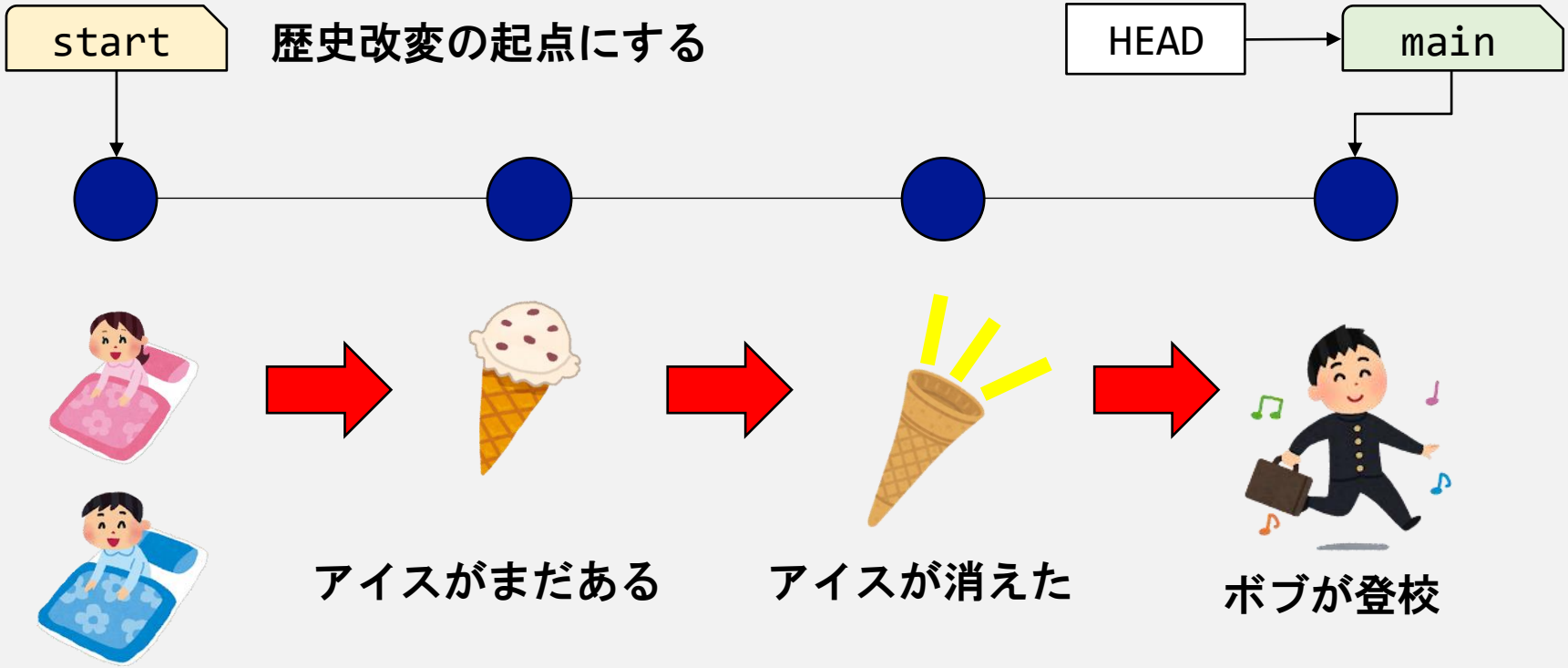


二人が目を覚ます

課題3 – Step 3

ブランチの作成

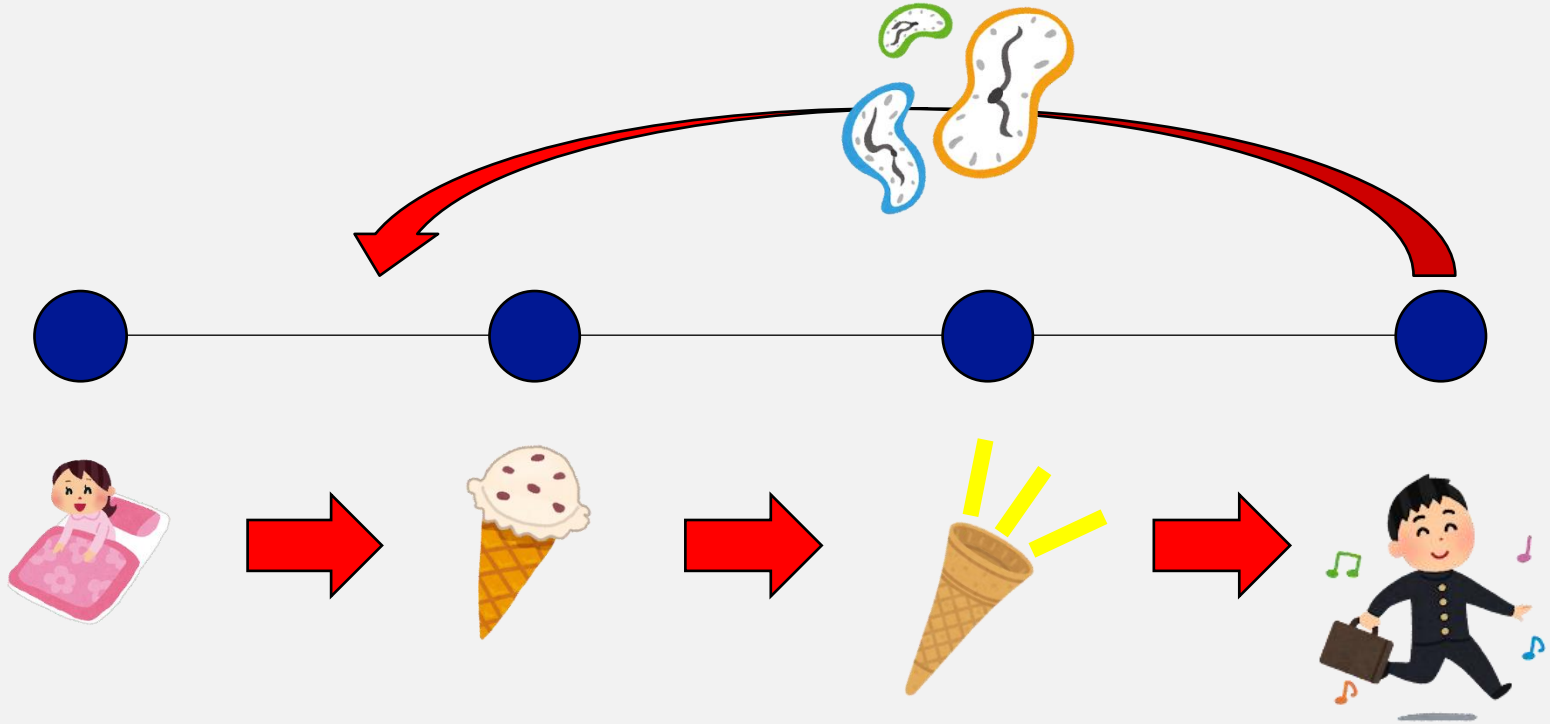
```
git branch start origin/start
```



課題3 – Step 4

歴史改変

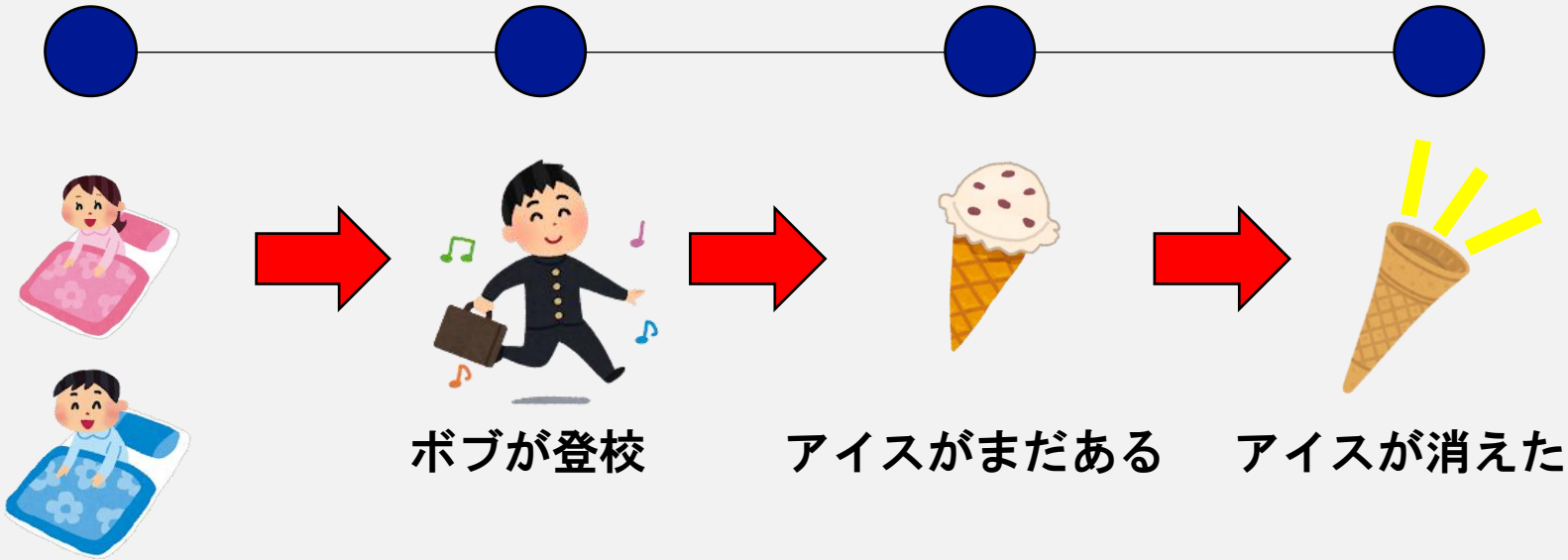
```
git rebase -i start
```



課題3 – Step 5

改変された歴史の確認

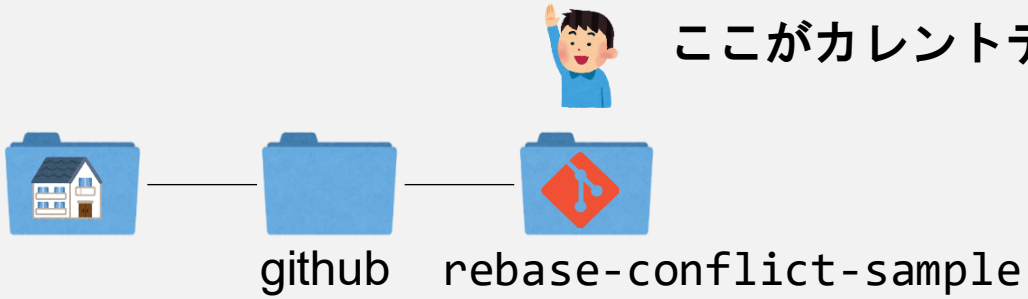
```
git log --oneline
```



課題4 – Step 1

リポジトリのクローン

```
cd
cd github
git clone https://github.com/cpss2026-git/rebase-conflict-sample.git
cd rebase-conflict-sample
```

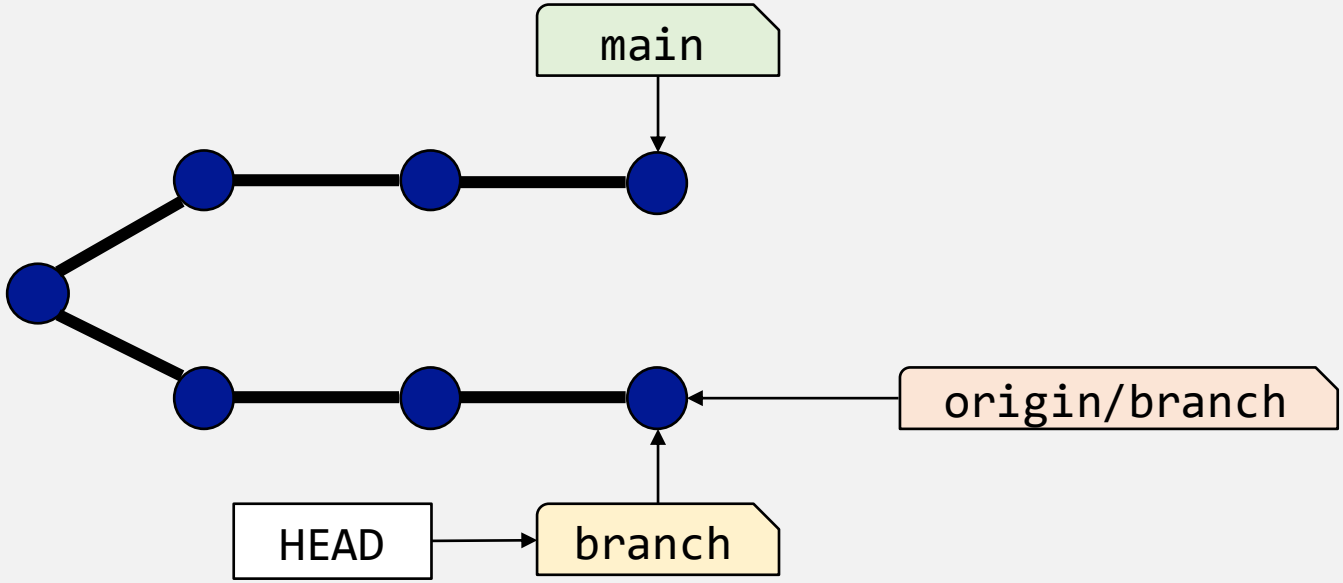


課題4 – Step 2

ブランチの準備

```
git switch -c branch origin/branch
```

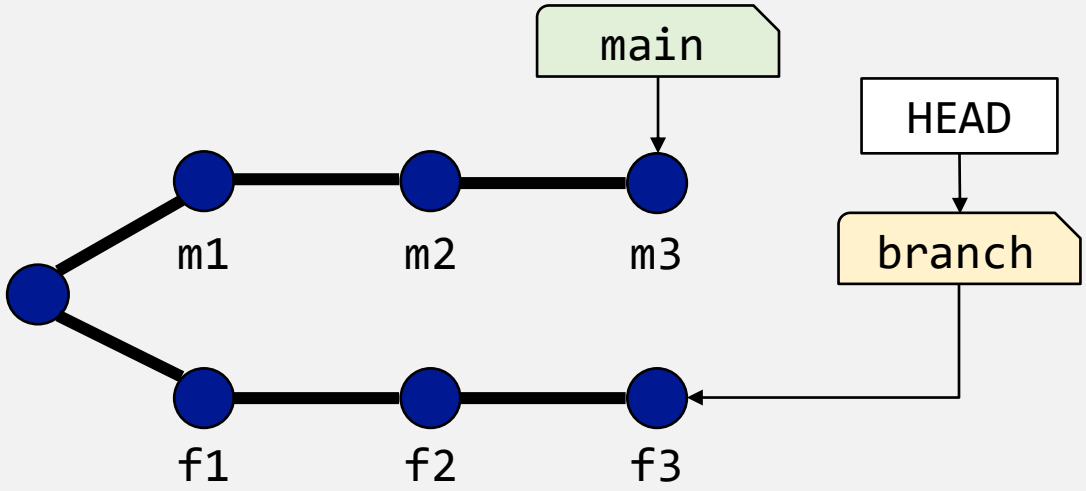
origin/branchの指すコミットにbranchというブランチを作成し、
カレントブランチをそこへ移動



課題4 – Step 3

歴史の確認

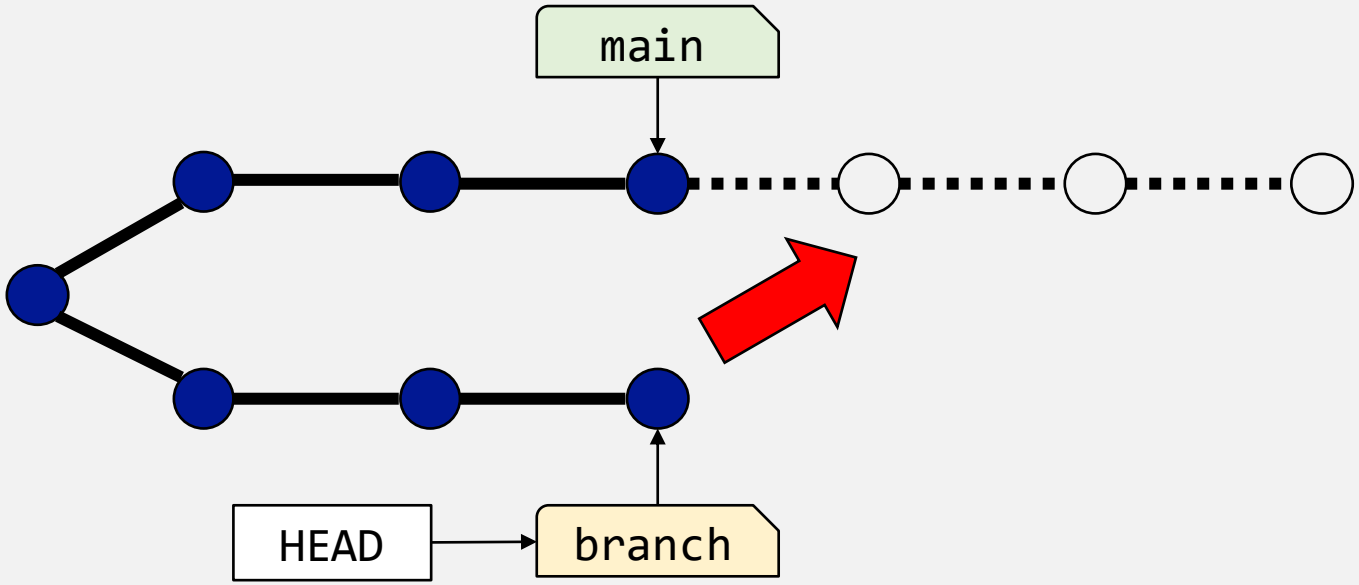
```
git log --all --graph --oneline
```



課題4 – Step 4

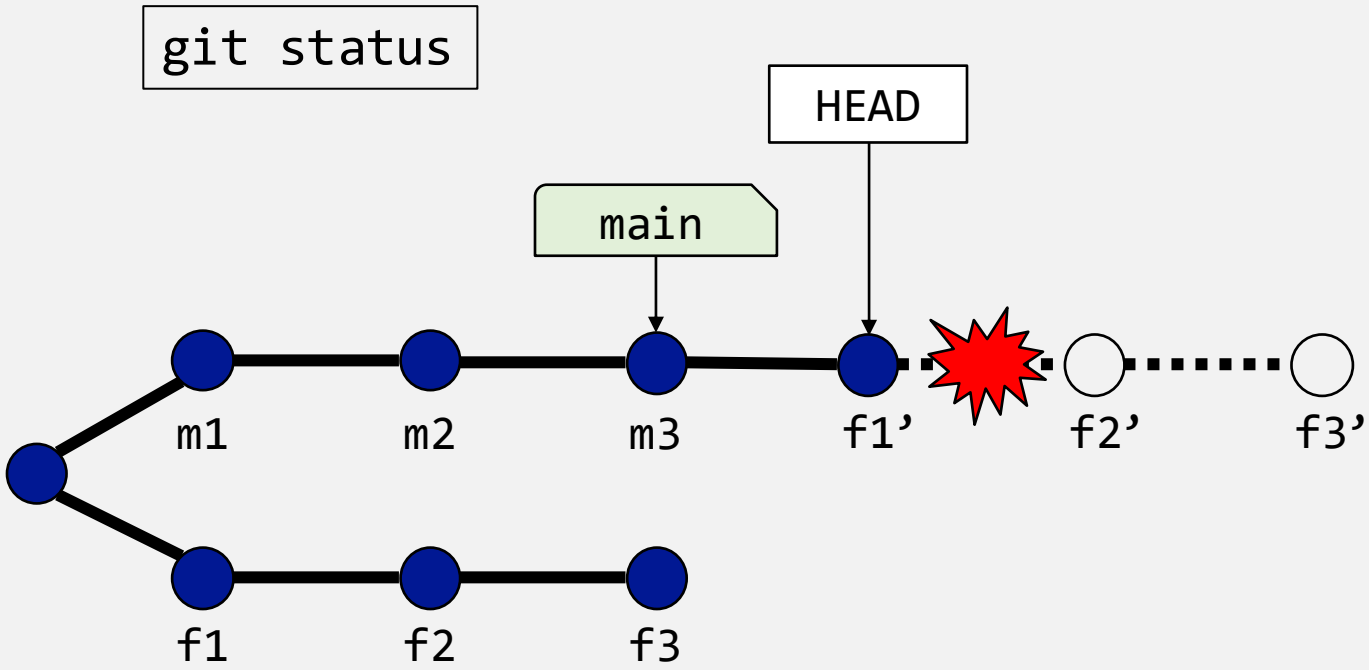
リベースの実行

```
git rebase main
```



課題4 – Step 5

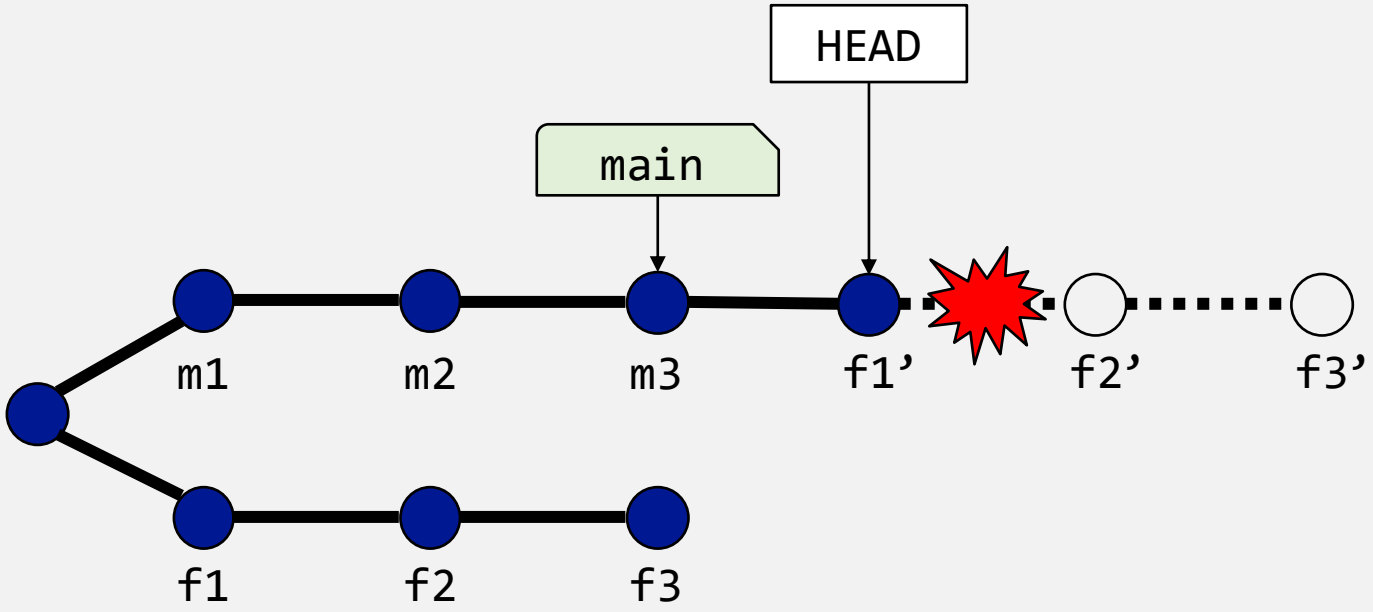
状態の確認



HEADは「最後にリベースがうまくいったコミット」を指している
(detached HEAD)

課題4 – Step 6

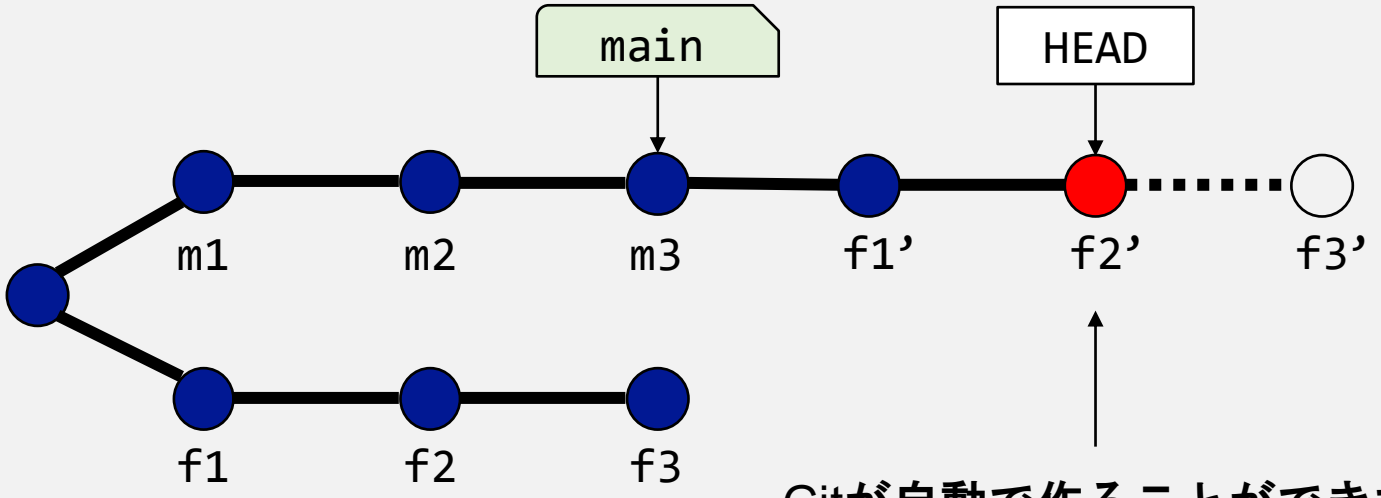
エディタで衝突状態にあるファイル「text1.txt」を修正し、保存する



課題4 – Step 7

解決をGitに伝える

```
git add text1.txt
git commit -m "f2"
```

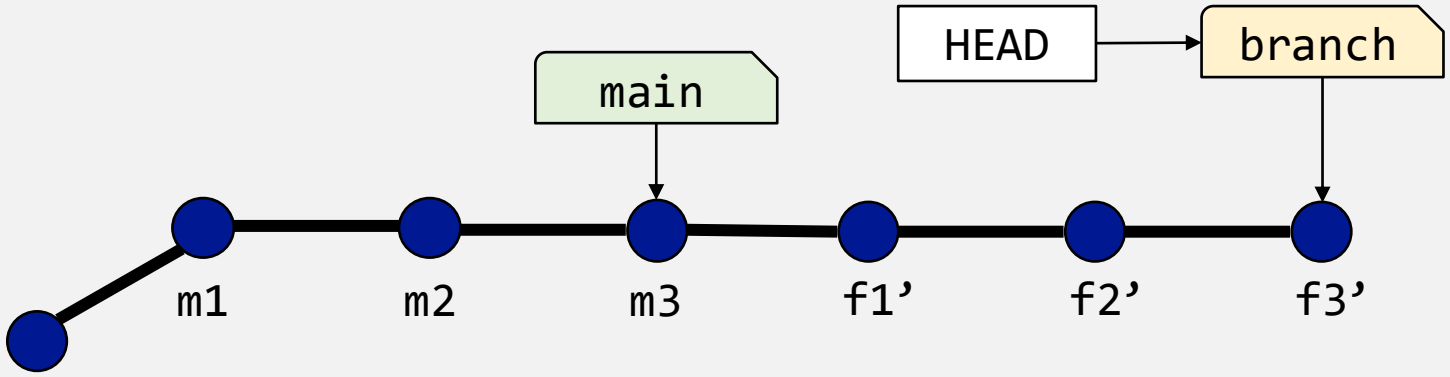


Gitが自動で作ることができなかった
このコミットを手動で作った

課題4 – Step 8

リベースの続行

```
git rebase --continue
```

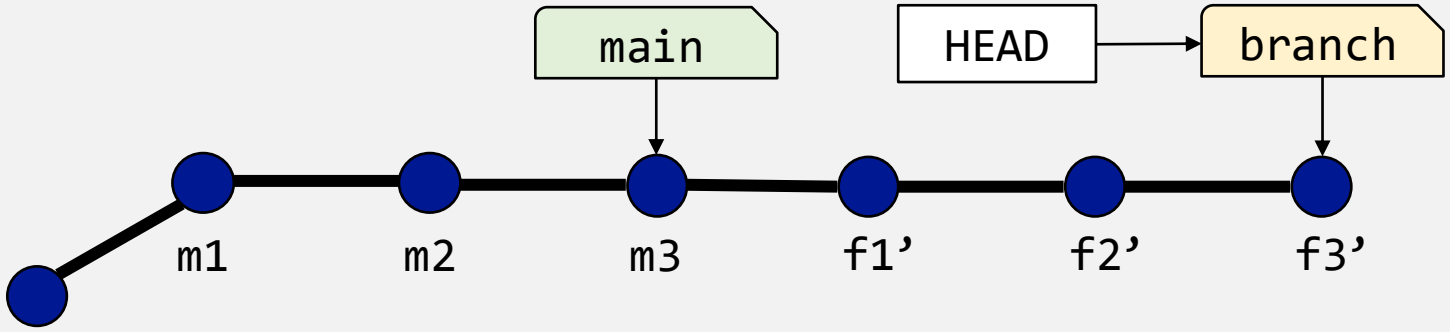


リベースが最後まで実行され、detached HEAD状態が解消される

課題4 – Step 9

歴史の確認

```
git log --oneline --graph
```

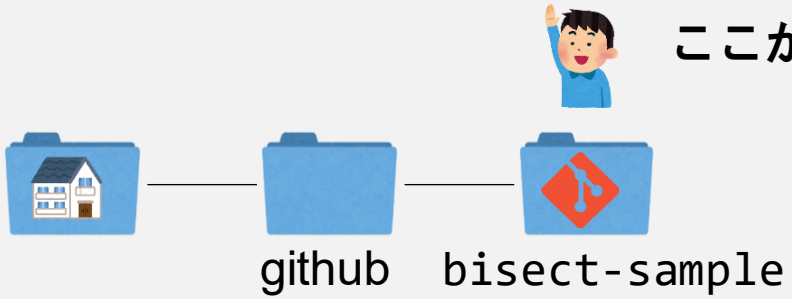


もともと分岐していた歴史が一本道になり、
mainからbranchへfast-forwardマージが可能になった

課題5 Step 1

リポジトリのクローン

```
cd
cd github
git clone https://github.com/cpss2026-git/bisect-sample.git
cd bisect-sample
```



課題5 Step 2

evenodd.sh

与えられた整数が偶数か奇数かを判定するコード

想定する動作

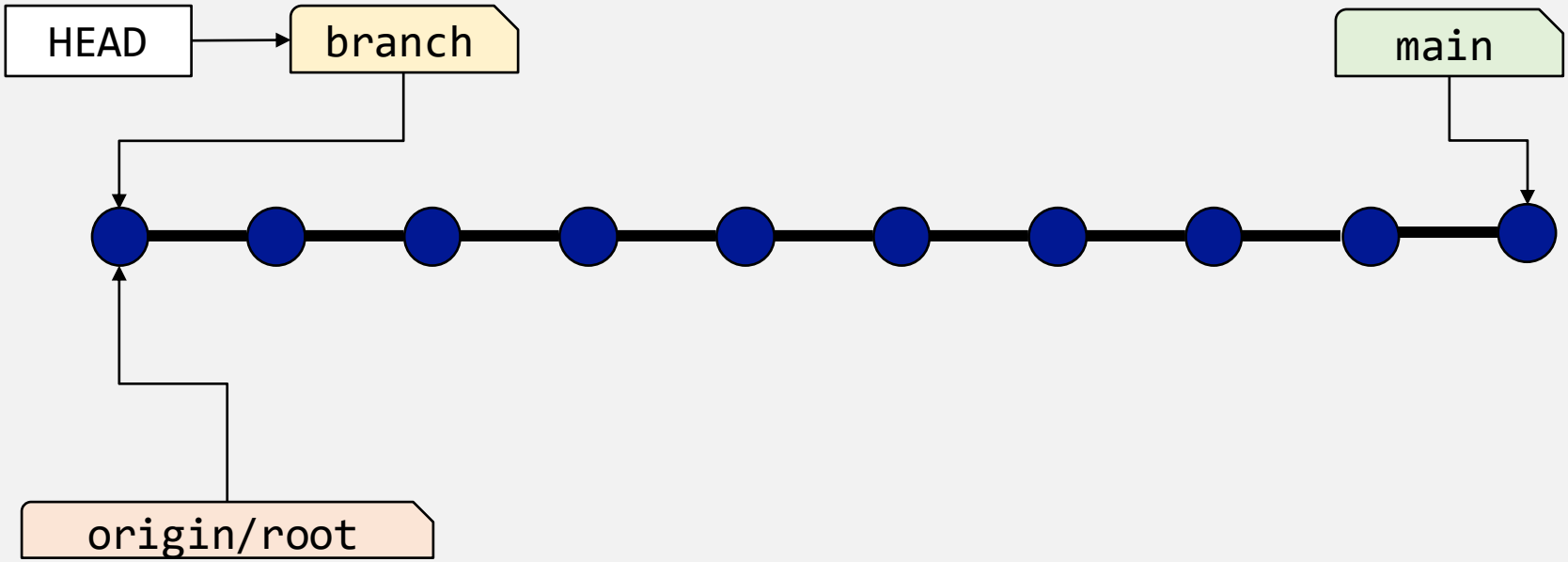
```
$ ./evenodd.sh 1
odd
$ ./evenodd.sh 2
even
```

実際の出力を確認し、バグがあることを確認

課題5 Step 3

origin/rootからrootブランチを作成し、カレントブランチをrootに変更

```
git switch root
```



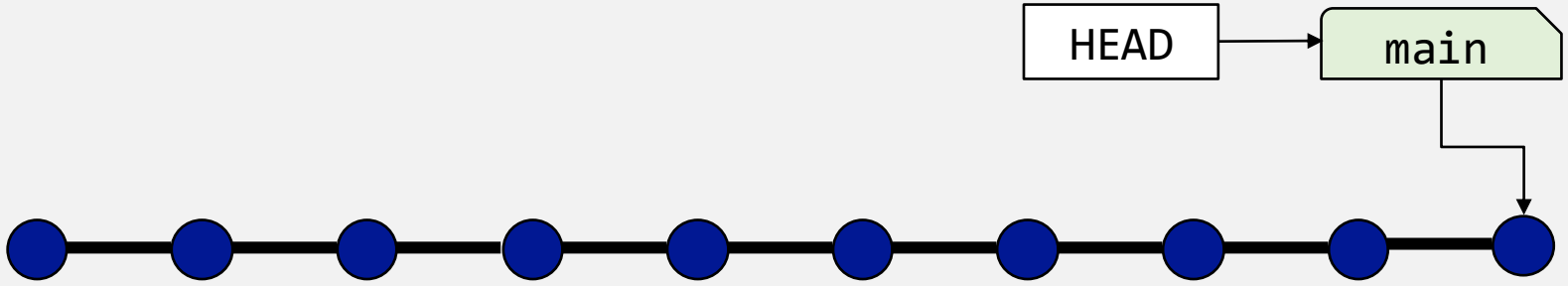
課題5 Step 4

動作が正しいことを確認

```
$ ./evenodd.sh 1
odd
$ ./evenodd.sh 2
even
```

カレントブランチをmainに戻す

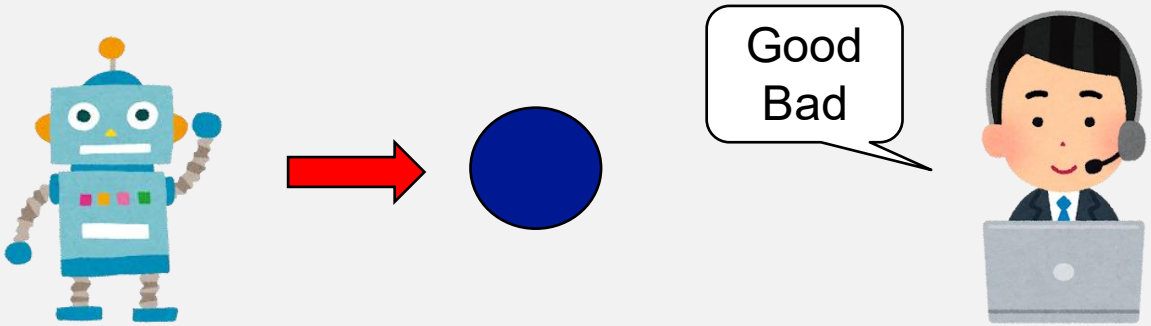
```
git switch main
```



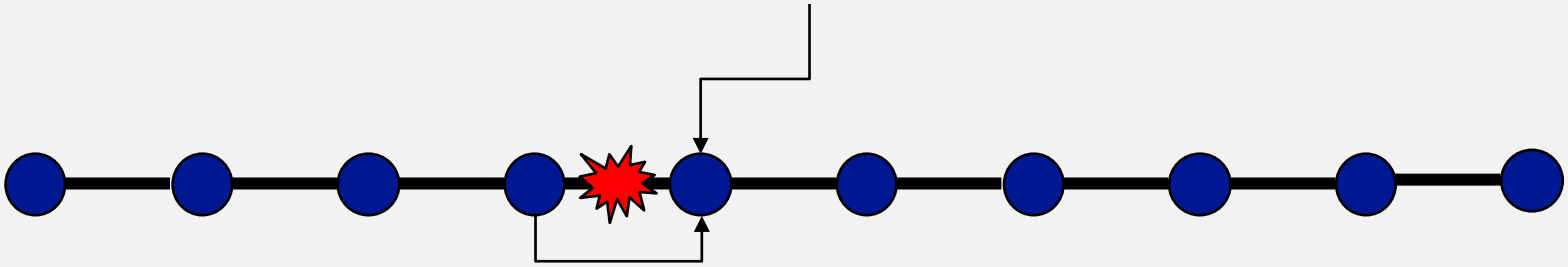
課題5 Step 5-7

git bisectの実行

```
git bisect start main root
```



はじめてバグが入ったコミット

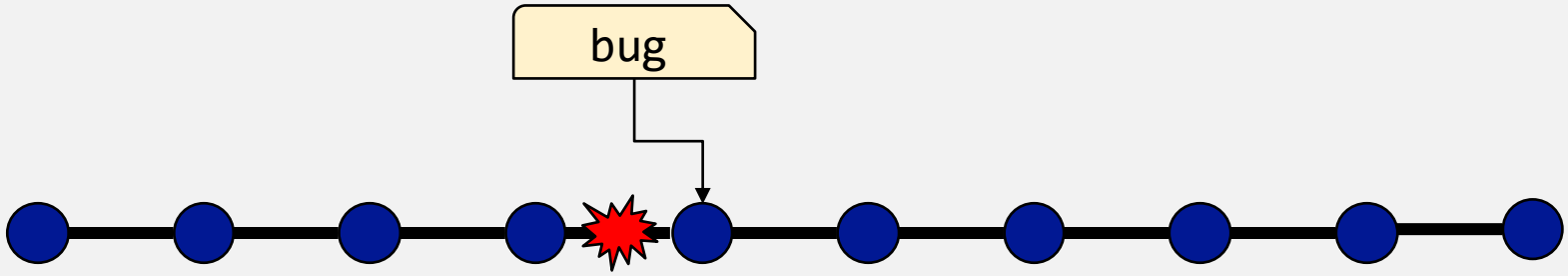


このタイミングでバグ挿入

課題5 Step 8

バグが入った直後のコミットにブランチを付与

```
git branch bug コミットハッシュ
```



git bisect モードを抜ける

```
git bisect reset
```

課題5 Step 9

スクリプトを用いた自動二分探索

test.sh

evenodd.shをテストし
動作が正しければ終了コード0を
正しくなければ1を
返すテストスクリプト

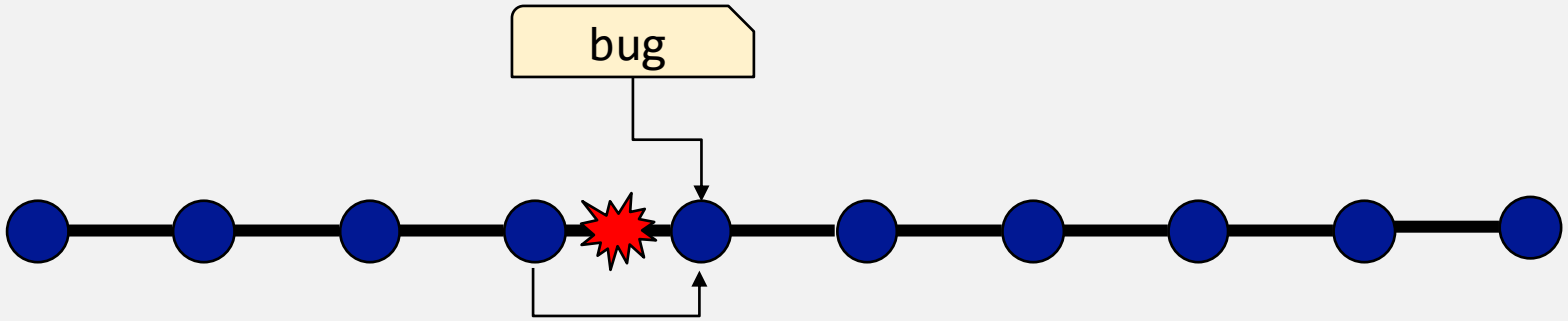
test.shを利用して自動で二分探索を実施

```
git bisect start main root
git bisect run ./test.sh
```

課題5 Step 10

バグが初めて入ったコミットにスイッチ

```
git switch bug
```



この二つのコミットの差分を調べる

```
git diff HEAD^
```